This is true, and in some cases the best thing to do is to make the generation of the validation check and response automatic. In fact, this is exactly the approach we will use for web APIs using MVC controllers with the `[ApiController]` attribute when we cover them in chapter 20.

For Razor Pages apps, however, you typically still want to generate an HTML response, even when validation failed. This allows the user to see the problem and potentially correct it. This is much harder to make automatic.

For example, you might find you need to load additional data before you can redisplay the Razor Page, such as loading a list of available currencies. That becomes simpler and more explicit with the `ModelState.IsValid` pattern. Trying to do that automatically would likely end up with you fighting against edge cases and workarounds.

Also, by including the `IsValid` check explicitly in your page handlers, it's easier to control what happens when additional validation checks fail. For example, if the user tries to update a product, the `DataAnnotation` validation won't know whether a product with the requested ID exists, only whether the ID has the correct format. By moving the validation to the handler method, you can treat data and business rule validation failures in the same way.

TIP You can also add extra validation errors to the collection, such as business rule validation errors that come from a different system. You can add errors to `ModelState` by calling `AddModelError()`, which will be displayed to users on the form alongside the `DataAnnotation` attribute errors.

I hope I've hammered home how important it is to validate user input in ASP.NET Core, but just in case: VALIDATE! There, we're good. Having said that, performing validation only on the server can leave users with a slightly poor experience. How many times have you filled out a form online, submitted